Underworld and Docker (part 1)

Julian Giordani[1]

[1]University of Sydney

DOI https://doi.org/10.6084/m9.figshare.33193329

Keywords AuScope UW Cloud, #editors-pick

PUBLISHED Sep 01, 2015 ARTICLE UWTN 2015-002 VERSION 1.0.0 LICENCE CC-BY-4.0 ARCHIVED 2026-08-10T04:19:43Z SOURCE https://www.underworldcode.org/underworld-and-docker-part-1/

While huge improvements in usability have been achieved in Underworld2, the installation process is still unfortunately as painful as ever. This difficulty is in large part due to Underworld's numerous dependencies, and also the multiple platforms we try to support. Compounding this, the legacy of a individual user's machine often conspires against success, rarely in obvious ways. Simply, compiling is not fun.

Enter docker.

1. What are docker containers ?

Essentially, docker containers allow developers to create completely self contained, ready to use, portable applications. They are similar to virtual machines, but there are a number of key differences that make the docker system *lighter* than traditional VMs. In particular, if you are using Linux, a docker container (i.e. a running docker instance) will directly leverage the existing operating system (live in memory), so there is no need for a secondary virtual OS (and the overhead it brings). For users on Mac OS X or Windows, some form of VM is required to run in the background for the docker instance to utilise, though this is setup and launched automatically.

The other key benefit is the Docker Hub ecosystem. This service provides two key functions:

Automated Builds: Whenever a change is made to the Underworld repositories, this triggers the generation of a new docker image build at Docker Hub. The build is created by taking a fresh Linux image (Debian), installing Underworld's dependencies, and then downloading and compiling Underworld.

Image Publication: Once the automated build process is complete, the fully baked image is made available at Docker Hub. Users can obtain this image directly via the command line (see below). This allows users to obtain the very latest development version of the code immediately, and to update trivially. Stable versions of the code will also be made available (soon), and can easily be selected at the command line.

Our Docker Hub page is here: https://hub.docker.com/u/underworldcode/

2. Getting setup for dockers

Installation instructions for various platforms is provided at Docker Hub: http://docs.docker.com/engine/installation/

3. Using Underworld dockers

We are ready to use our Underworld docker.

Windows & OS X: Make sure your *docker-machine* VM is up and running! The easiest way to start the *docker-machine* is to use the *Docker Quickstart Terminal*. This will launch the docker-machine VM (creating it first if necessary), and also setup various environment variables within the terminal. For Windows the *Docker Quickstart Terminal* requires that virtualization be enabled, which can be done in the BIOS settings when you boot your machine.

Now let's run Underworld. We will launch a new docker container using the `docker run` command:

```
$ docker run -p 8888:8888 underworldcode/underworld2
```

This command will create a new *container* using the underworldcode/underworld2 image. Note that it will first check to see if the image exists locally, and otherwise will download a copy from the Docker Hub. This will only happen the first time you run the command; subsequent execution will use the downloaded image. Once the new instance is created, Jupyter notebooks is launched within the container. Note that we also pass an option of the form `-p host_port:docker_port` which tells docker to perform a port mapping from the docker instance to the host. This allows us to use our native web browser to access the active docker notebook instance at http://localhost:8888/ (Windows & OS X users see below).

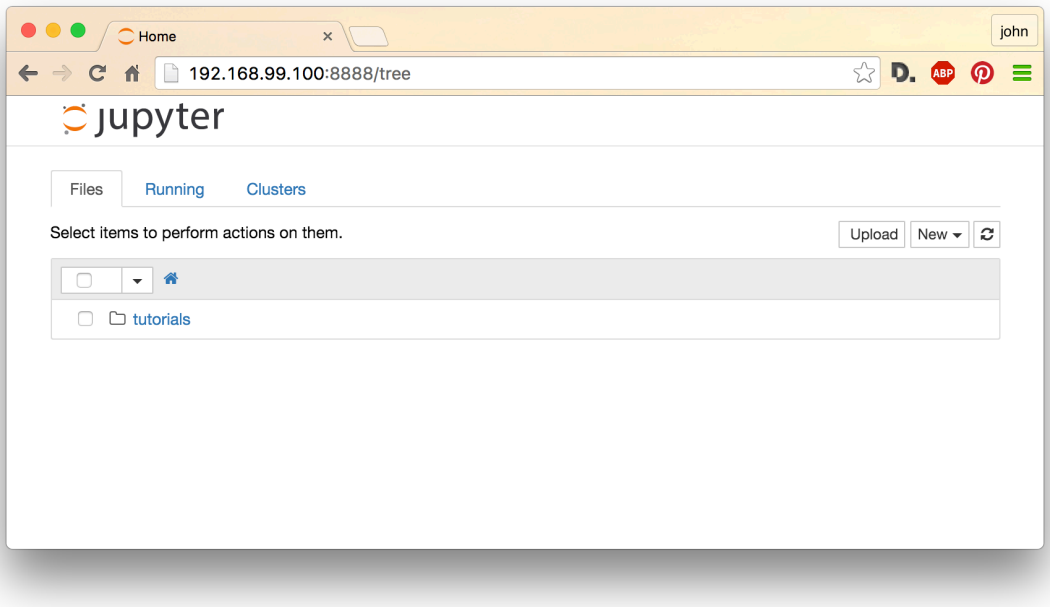

Figure 1: *Jupyter notebook running inside a docker container on OS X*

Windows & OS X: Note that you are actually running your dockers within the *docker-machine* VM, so you need to browse to the address of the VM **not** localhost. In the example above you would open http://192.168.99.100:8888/ in your browser where the ip address is 192.168.99.100. To find out the address of your *docker-machine*, you should run `docker-machine ip default` from within your Docker Quickstart Terminal. Here we've assumed that your *docker-machine* is named 'default' (which in most cases it should be), but if that does not seem to be working, you can determine the name of your *docker-machine* by running `docker-machine active`.

4. Containers and Images

It is important to understand the difference between these two docker objects.

Containers: Containers are a form of virtualisation. Unlike traditional virtual machines, containers do not encapsulate the entire operating system, only a smaller layer on top of the base operating system. A Docker container is simply a particular implementation of this concept.

Images: Images are essential snapshots of containers at a particular instance. By taking a snapshot of a container, developers are able to completely encapsulate the environment necessary to run an application. Images are the objects published to the Docker Hub. They are read-only.

When a user executes `docker run`, effectively they are creating a **new** container, the starting point of which is the docker image originally obtained from the Docker Hub. This container may be used ephemerally (with execution effectively restarting from the image each time), or may be retained and modified (as is more typical for traditional VM use). For Underworld dockers, ephemeral usage is usually the most natural choice.

You may convince yourself of these details using the `docker images` and `docker ps` commands. The former command will list the docker images you have available locally, while the latter will list all your containers. If you have already ran an Underworld2 docker (as we did earlier using the `docker run` command), you should have a local copy of the underworldcode/underworld2 image:

```
$ docker images
REPOSITORY                  TAG                 IMAGE ID            CREATED             VIRTUAL
SIZE
underworldcode/underworld2  latest              b6421851238e        8 hours ago         1.147 GB
```

Likewise, you will have created a new container for each time you used `docker run`:

```
$ docker ps -a
CONTAINER ID        IMAGE                         COMMAND                  CREATED
STATUS                      PORTS                    NAMES
5e157f9a7f09        underworldcode/underworld2    "/usr/local/bin/tini "   2 seconds ago       Up 1
seconds                0.0.0.0:8888->8888/tcp   awesome_mietner
69f8ad2e4988        underworldcode/underworld2    "/usr/local/bin/tini "   6 seconds ago
Exited (0) 3 seconds ago                            romantic_wing
98e69734bb38        underworldcode/underworld2    "/usr/local/bin/tini "   10 seconds ago
Exited (0) 7 seconds ago                            distracted_bell
196b061e87d8        underworldcode/underworld2    "/usr/local/bin/tini "   26 seconds ago
Exited (0) 10 seconds ago                           suspicious_hugle
916f8761c9ee        underworldcode/underworld2    "/usr/local/bin/tini "   31 minutes ago
Exited (0) 31 minutes ago                           small_colden
```

Note that we used the `-a` option which lists **all** containers, including those which are no longer running. From the `STATUS` column, we can see that only a single container is currently running. The `IMAGE` column shows which image the container was derived from. Exited containers may be restarted using the `docker start` command, while you may remove containers you no longer require using `docker rm`. Similarly, redundant images may be deleted using `docker rmi`.

5. Launching Underworld docker containers

Let's take a closer look at the `docker run` command:

```
$ docker help run
Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Run a command in a new container

  -d, --detach=false              Run container in background and print container ID
  -i, --interactive=false         Keep STDIN open even if not attached
  -p, --publish=[]                Publish a container's port(s) to the host
  --rm=false                      Automatically remove the container when it exits
  -t, --tty=false                 Allocate a pseudo-TTY
  -v, --volume=[]                 Bind mount a volume
```

Only the more immediately useful options are listed above. For the full list, run `docker help run`.

Returning to the earlier example,

```
$ docker run -p 8888:8888 underworldcode/underworld2
```

you may note that we selected the `underworldcode/underworld2` image, but no command was specified to run in the container. In this case, a default command is instead executed. This default is specified by the developer at image creation time. For the Underworld docker, the default behaviour is to launch Jupyter notebooks via the `jupyter notebook` command, and so the above is equivalent to

```
$ docker run -p 8888:8888 underworldcode/underworld2 jupyter notebook
```

The instances created above come prepackaged with Underworld tutorial notebooks which reside within the docker containers, but what if you want to run your own notebooks or python scripts? Well it is possible to upload your models via the Jupyter notebooks web interface, but this far from ideal. A better approach is to use the `-v` option to bind a local (ie host) directory to a directory within the container. In effect, this will allow your docker instance to seamlessly act upon files which reside on the host machine.

```
$ docker run -p 8888:8888 -v $PWD:/workspace/my_data underworldcode/underworld2
```

The volume mapping binding option takes the form `-v host_directory:docker_directory`. In this example, we map the current execution directory to the `/workspace/my_data` directory within the docker. Note that we have used the `PWD` environment variable as a shortcut to the current directory absolute path. You should find that the running notebook instance now displays the files that exist in the current host directory. Any file modification and/or creation will be recorded to the host machine (in the current execution directory).

Direct python (non-notebook) usage is also very straightforward:

```
$ docker run -v $PWD:/workspace/my_data underworldcode/underworld2 python YourLocalScript.py
```

In this instance, the generated container will run the command `python YourLocalScript.py`. Note that for the script `YourLocalScript.py` to be available inside the container, we still need to use the `-v` option to make the local directory accessible to the container.

As a final usage example, we will run an interactive docker container by simply launching a bash shell:

```
$ docker run -v $PWD:/workspace/my_data -p 8888:8888 -i -t underworldcode/underworld2 /bin/bash
```

For interactive usage, it is necessary to include the `-i` and `-t` options. You should find yourself at the prompt of a bash shell inside the Underworld docker container. We have still include both the directory mapping (to access files on the host), and the port mapping (should you wish to launch a notebook). The Underworld installation may be found at `/root/underworld2`, while as per your volume mapping, your host files should be accessible at `/workspace/my_data`. Note that you are the *root* user, so have full privileges, but don't have to worry about breaking things as you can simply discard the current container and start again! The Underworld docker image is based off a Debian distribution, so `apt-get` may be used to install any further packages (remember to first run `apt-get update`).

6. Obtaining different versions of Underworld

Developers publishing dockers may tag particular images to correspond to a particular code release. To select a tagged docker image, simply postfix the docker image name with the required tag. For example, run a container based off Ubuntu 14.04

```
$ docker run ubuntu:14.04
```

Note that if no tag is specified, the *latest* tag will be accessed which corresponds to the latest image build. Underworld2 does not have any tagged stable releases yet, so all docker usage will correspond to the latest development version of the code. Once available, stable tagged images will be recommended for most usage, though the ability to trivially switch to different tagged images allows users to easily experiment with newer releases or development versions.

Remember that `docker run` will first check locally for the requested image. Users may need to occasionally update their local image to incorporate recent bug fixes (if using a tagged release image), or to obtain the very latest Underworld developments (for bleeding edge users). Updates are affected using the `docker pull` command:

```
$ docker pull underworldcode/underworld2
```

7. MPI and docker

Your Underworld simulations may be executed in parallel by simply invoking `mpirun` as usual, though the MPI execution must occur *within* the container:

```
$ docker run -v $PWD:/workspace/my_data underworldcode/underworld2 mpirun -np 2 python
YourLocalScript.py
```

Windows & OS X: You may need to ensure your *docker-machine* is configured to allow access to multiple CPU cores. Note that the standard Docker setup leverages VirtualBox for virtualisation, so you will need to access the VirtualBox configuration panel.

While not currently applicable to large scale parallel simulation, early research into using dockers across HPC facilities shows promise.

8. Other hints

Windows & OS X: If you are encountering difficulties running docker commands, restarting your *docker-machine* (using `docker-machine restart default`) may be necessary.

Windows: You may need to turn the firewall off when installing the *Docker Quickstart Terminal*. Note that some issues have been encountered with using the alpha version of *Kitematic* for Windows 7, hopefully they will fix these in the full release. In the meantime the *Docker Quickstart Terminal* does work in Windows 7 using the instructions above.

9. Further reading

- https://www.docker.com/what-docker
- https://en.wikipedia.org/wiki/Docker